WaveMAX External Domain Security Audit — 2026-04-30

External Domain Security Audit — 2026-04-30

Subject domains: `wavemaxlaundry.com` (franchise mail tenant) and `mhrmarketing.com` (vendor marketing/intake/payment site) **Trigger:** internal account takeover phishing at `wavemaxlaundry.com` correlated in time with a marketing rollout asking franchisees to use `mhrmarketing.com` for payment + intake. **Author:** Rick Houlihan, with analysis support **Status:** active incident; remediation pending

---

TL;DR

The phishing email is **NOT** a spoof — it's an **account takeover (ATO)** of `pk@wavemaxlaundry.com` inside the Microsoft 365 tenant. The headers prove the message was authored from a real authenticated session inside the tenant. The `mhrmarketing.com` site is timing-correlated but is not itself the source of the phish; it is, however, a sloppy attack surface that **could plausibly be the credential-harvesting vector** that compromised PK's account, and is unsafe in its own right.

---

1. Phishing Email — Forensic Read

The headers from the captured "OneDrive shared file" message are unambiguous:

Header	Value	What it means
`X-MS-Exchange-Organization-AuthAs`	`Internal`	Tenant treated this as fully trusted internal mail
`X-MS-Exchange-Organization-AuthMechanism`	`04`	SMTP submission with credentials (a real password was used)
`X-MS-Exchange-Organization-AuthSource`	`PH7PR14MB5641.namprd14.prod.outlook.com`	M365 mailbox server in this tenant
`X-MS-Exchange-CrossTenant-Id`	`14e70ab3-eab8-4a93-ab61-b7fec3877261`	Tenant ID — sender and recipient both inside it
`Authentication-Results`	`dmarc=none action=none`	DMARC bypassed because the message is internal
Body	TNEF (`winmail.dat`) + zero-width-char "OneDrive" header	Standard Outlook ATO phish payload designed to evade text filters

Verdict: the attacker has working credentials for `pk@wavemaxlaundry.com` and is sending "OneDrive shared file" lures from PK's real mailbox to harvest more credentials inside the franchise network. This is the canonical M365 worm pattern.

The timing correlation with the `mhrmarketing.com` rollout is suggestive, not conclusive — see §5.

---

2. wavemaxlaundry.com — Actual State

```
NS:    ns49/50.domaincontrol.com    (GoDaddy)
A:     160.153.0.155                (GoDaddy parking/hosting)
MX:    *** NONE PUBLISHED ***       (mail flows via RFC 5321 A-record fallback to GoDaddy → M365)
SPF:   *** NONE ***                 (anyone can spoof from:@wavemaxlaundry.com)
DKIM:  *** NONE ***                 (no selector1/selector2 CNAMEs to M365)
DMARC: v=DMARC1; p=quarantine; rua=mailto:dmarc_rua@onsecureserver.net
M365:  Federated via sso.godaddy.com (GoDaddy's resold M365)
```

Critical issues

1. **No SPF record.** External attackers can send `From: ceo@wavemaxlaundry.com` and pass basic checks. DMARC quarantines, but with nothing to align against, even legitimate mail likely lands in junk.
2. **No DKIM.** M365 will sign with `selector1/selector2._domainkey` if the CNAMEs are published. They aren't.
3. **No published MX.** External mail is delivered only via **RFC 5321 §5.1 A-record fallback** — the sending MTA finds no MX, falls back to the A record at `160.153.0.155` (GoDaddy), which relays into the M365 tenant. This is a GoDaddy-resold-M365 quirk, not a normal config. Many large senders (Google, Microsoft consumer, anti-abuse-strict gateways) **do not** fall back anymore — RFC made it optional and modern senders treat "no MX" as "deliberate no-mail." **You are silently losing some inbound mail and don't know which.**

4. **DMARC reports go to `onsecureserver.net`** (a GoDaddy-resold service). They almost certainly aren't being read — every spoof attempt and every internal compromise has been silently aggregated there.
5. **Federation through GoDaddy SSO.** Auth happens at `sso.godaddy.com`, not Microsoft. MFA enforcement, conditional access, and impossible-travel detection are limited by GoDaddy's identity tier — typically much weaker than native Entra ID Premium.
6. **AuthMechanism 04 (SMTP basic auth)** worked for the attacker. Even with MFA on the web login, legacy SMTP AUTH/IMAP/POP usually bypass MFA unless explicitly disabled.
7. **MX-bypass for inbound.** Without an explicit MX, attackers know any sender that *does* fall back will hit the A record directly. Combined with no SPF, this is a clean spoof + replay channel.

---

3. wavemaxlaundry.com — Lockdown Plan (priority order)

Within 1 hour (incident response)

1. **Force password reset on `pk@wavemaxlaundry.com`** and revoke all tokens/sessions in Entra ID admin center.
2. **Audit PK's mailbox for inbox rules and forwarding rules.** Attackers always plant rules like "move replies from IT to RSS Subscriptions". Check Outlook → Settings → Mail → Rules **and** Forwarding **and** OAuth-granted apps under "Connected apps".
3. **Pull sign-in logs for PK** for the last 30 days from Entra. Look for foreign IPs, unusual user agents, and successful basic-auth/SMTP-AUTH events.
4. **Scan all other internal recipients of the phish.** Anyone who clicked "Open" on the OneDrive lure is potentially next. Force password resets on them too.
5. **Audit all admin-role accounts in the tenant** (Global, Exchange, SharePoint admins). If any were targets, treat as compromised.

Within 24 hours (close the door)

6. **Disable Basic Auth/Legacy Auth tenant-wide.** In Exchange Online: disable SMTP AUTH, IMAP, POP, MAPI, ActiveSync as needed. This single change blocks the most common ATO replay vector.

7. **Enable Security Defaults or build a Conditional Access policy** requiring MFA for all users, all locations, with no legacy-auth exceptions.

8. **Publish SPF:**

   ```
   v=spf1 include:spf.protection.outlook.com include:secureserver.net -all
   ```

   (Drop `secureserver.net` once mail flow has been moved off GoDaddy.)

9. **Publish DKIM** by enabling it in the Microsoft 365 Defender portal and publishing the two `selector1/selector2._domainkey` CNAMEs to DNS.

10. **Tighten DMARC** to: `v=DMARC1; p=quarantine; pct=100; sp=quarantine; adkim=s; aspf=s; rua=mailto:<your-real-mailbox>@wavemaxlaundry.com; ruf=mailto:<your-real-mailbox>@wavemaxlaundry.com;` and **verify reports actually arrive.** Move to `p=reject` after 2 weeks of clean reports.

11. **Publish an explicit MX.** Go to M365 admin **Settings → Domains → wavemaxlaundry.com → DNS records** and use the literal MX hostname Microsoft provides for *this* tenant. For GoDaddy-resold M365 it is typically:

    ```
    wavemaxlaundry.com.  3600  IN  MX  0   smtp.secureserver.net.
    wavemaxlaundry.com.  3600  IN  MX  10  mailstore1.secureserver.net.
    ```
    For migrated-direct-to-Microsoft, it is:

    ```
    wavemaxlaundry.com.  3600  IN  MX  0   wavemaxlaundry-com.mail.protection.outlook.com.
    ```

Within 1 week (structural)

12. **Move identity off GoDaddy SSO.** Federate directly against Entra ID (managed domain) and decommission `sso.godaddy.com` as the auth point. GoDaddy's resold M365 stack is consistently weaker on identity controls than buying licenses directly from Microsoft.
13. **Enable Microsoft Defender for Office 365 Plan 1+** (Safe Links, Safe Attachments, anti-phish impersonation rules naming VIPs).
14. **Mandatory phishing-resistant MFA** (Authenticator number-match minimum; FIDO2 keys for admins).
15. **Publish MTA-STS and TLS-RPT** records to harden inbound TLS.
16. **Enable mailbox audit logging** if not already on, and forward to a SIEM or at minimum to Sentinel for free 90-day retention.

---

4. mhrmarketing.com — Risk Summary

Context: MHR Marketing has been granted rights to produce the entire new WaveMAX web presence. They are not just a marketing brochure vendor — they are the franchise's outsourced web team. Every weakness below is a weakness in WaveMAX's customer-facing surface area.

Stack identified (full)

- **Hosting:** GoDaddy Managed WordPress (`160.153.0.197`, `x-gateway-cache-*` headers) behind Cloudflare.
- **CMS:** WordPress 6.9.4, **Divi theme v4.27.6**, single non-relevant plugin (`contact-widgets`). **No form plugin, no security plugin, no caching plugin** beyond what GoDaddy bundles.
- **Identity (MHR's own M365):** the SharePoint URL in the email is `netorgft2017901.sharepoint.com` — the `netorgft####` prefix is GoDaddy's automated tenant naming for resold M365. **MHR Marketing is on the same GoDaddy-resold M365 stack as wavemaxlaundry.com**, which means they share the identity-tier weaknesses (legacy SMTP AUTH, GoDaddy-federated SSO, no Entra Premium controls). If their account is phished, all franchisee photos and intake data uploaded to that SharePoint are at risk in exactly the same way PK's mailbox is.
- **Payment page (`/payment/`):** Stripe Pricing Table widget (`prctbl_1TJM6nKEqytXvjrU9X3iUyBq`, `pk_live_51SzOD8KEqytXvjrU…`) + three Stripe-hosted Payment Links (`buy.stripe.com/5kQ7sM…`, `bJeeVe…`, `fZu5kE…`). Card data never touches the WordPress site.
- **Intake page (`/wavemax-intake/`): completely custom client-side JavaScript form** (107 lines of inline JS) backed by a Google Apps Script `/exec` endpoint that almost certainly writes to a Google Sheet. Full source is captured in §6.
- **Customer/franchisee portal (`portal.mhrmarketing.com`): GoHighLevel SaaS-Mode whitelabel.** Confirmed by `class="lead-connector"`, `static.leadconnectorhq.com`, `gohighlevel.com` origin trials, `pendo` analytics, and the `*.ludicrous.cloud` CNAME chain (`ludicrous.cloud` is the agency's whitelabel brand of GoHighLevel). The HTML loads **TinyMCE 4.9.11**, a deprecated 2020-era version with known XSS history.
- **`api.mhrmarketing.com`:** Cloudflare bot-challenged backend, also part of the GoHighLevel whitelabel.
- **Mail authentication:** DMARC `p=none`, **no SPF, no DKIM** (same hole as wavemaxlaundry.com — anyone can spoof MeganR@mhrmarketing.com to franchisees).
- **WordPress admin:** `/wp-login.php` open with no IP allowlist; the WP REST API discloses **admin user `meganbreanne` (id=1)** and `?author=1` redirects to `/author/meganbreanne/` confirming the username. The login form is a brute-force target with a known username.

Specific implementation problems

These are the bugs and design errors I found by reading the actual page source and probing endpoints. They are not abstract risks — they are present in the running site today.

Intake form (`/wavemax-intake/`) — concrete defects

The full inline JavaScript that powers the intake form is captured in §6. Reading it directly:

1. **The submit method is `GET`, not `POST`.** The code reads `fetch(U+'?d='+encodeURIComponent(JSON.stringify(d)), {method:'GET',mode:'no-cors'})`. Every field (phone, address, contact email, owner name, owner bio, pricing, equipment counts, testimonials, Google Place ID, etc.) is concatenated as one big JSON blob and crammed into the URL query string. URLs are logged everywhere — browser history, ISP DNS resolvers (the hostname leaks even if HTTPS hides the path), corporate proxies, Cloudflare access logs, Google Apps Script execution logs, browser extensions, monitoring SDKs. All of those now have a copy of every franchisee's intake data. Standards-compliant POST with `Content-Type: text/plain` would have avoided this entirely; the developer chose GET because they couldn't get CORS to work and gave up.

2. **`mode: 'no-cors'` blinds the page to the response.** The server's reply is "opaque" by browser policy — the JS literally cannot read it. The `.then(function(){ ok.style.display='block'; modal.style.display='flex'; })` block runs whether the Apps Script succeeded, returned a 500, or doesn't exist. The user *always* sees "Submitted successfully!" The only way submissions can fail visibly is a network-layer error (DNS down, CORS preflight denied for some reason). **You have no idea how many franchisee submissions have silently been dropped.**

3. **Validation is one `alert()`.** The only check before submit is `if(!d.wavemax_name){alert('Please enter the WaveMAX location name.');return;}`. Anything else — invalid phone, malformed Google Maps URL, garbage in price fields, unicode in numeric fields, 10MB of text in `owner_bio` — sails through. The Apps Script presumably writes whatever it gets straight into a Google Sheet cell.

4. **The location dropdown ships a `slug-not-found` placeholder option.** Selecting `["slug-not-found", "⚠ Location Slug Not Found"]` and submitting still posts the form with `location_slug=slug-not-found`. The downstream system (whatever turns this Sheet into a website) presumably keys on `location_slug`. There is no protection against a franchisee leaving the placeholder selected and corrupting the dataset, or against a competitor submitting fake data tied to a real slug like `austin-tx`.

5. **The Apps Script endpoint ID is a public secret.** The hard-coded URL `https://script.google.com/macros/s/AKfycbzSEdMVk1UIe3710jIFNBgkBtJw2ZGa6CmDjCIQ2mmGPneKSU50zoR2EVci7fjNHN0/exec` is callable by anyone. It returns 302 to `https://script.googleusercontent.com/...` on GET, meaning it is deployed as "Anyone, even anonymous." A hostile actor with the URL can:

    - Flood the Sheet with thousands of fake rows (cost: zero).

    - Fill the dataset with offensive content embedded in `owner_bio` or `promo_bar_text`, which then gets rendered into the live customer-facing site.
    - Exhaust Google Apps Script quotas (6 hr/day execution time per account) and DoS legitimate submissions. There is no captcha, no rate limit, no auth header, no shared-secret token, no Origin check, no request signing.

6. **There is no payment gate.** The email body claims "a payment method must be on file in order to unlock your website form. Until that is completed, there will not be a form live on your website." That gate exists only as a hyperlink ordering. Technically, the intake form has no idea whether you've paid — there is no Stripe webhook, no customer ID lookup, no token check. Anyone who has the `/wavemax-intake/` URL (which is in the franchisee email and indexable by search engines) can submit data without ever visiting `/payment/`.

7. **No CSRF/Origin protection.** Apps Script's "Anyone, even anonymous" deployments do not enforce Origin or Referer. A malicious page on any domain can render an `<img src="https://script.google.com/macros/s/AKfy.../exec?d=...">` and submit data from a victim's browser. The only requirement is knowing the endpoint ID, which is now in the page source forever.

8. **The "Photo upload" step trusts an externally-owned SharePoint.** The success modal directs franchisees to `https://netorgft2017901.sharepoint.com/sites/ResourceCenter/Photos/Forms/PhotosGrouped.aspx`. That tenant is **MHR Marketing's** GoDaddy-resold M365 — outside WaveMAX's control. WaveMAX has no audit trail of who uploaded what, no enforced naming convention beyond a written instruction, and no recourse if MHR's account gets phished and the photos are exfiltrated or replaced.

Payment form (`/payment/`) — defects

1. The mechanics are sound (Stripe Pricing Table + Payment Links keep card data out of mhrmarketing.com — PCI scope is nil for the merchant). **But:**
2. **There is no logical link from `/payment/` back to `/wavemax-intake/`.** A successful Stripe Payment Link checkout sends the customer to a Stripe-hosted thank-you page (or a configured redirect — none observed). Nothing on the WordPress site verifies the payment status before showing the intake form. The "unlocked" state the email describes is a marketing claim, not a software gate.
3. **The Stripe customer record and the Google Sheet record share no key.** `wavemax_name` (free-text) is the join key in the Sheet. There is no Stripe `customer_id`, no `metadata.location_slug`, no idempotency reference. Reconciling who paid against who submitted will be manual forever.
4. **Three separate Payment Links** (`buy.stripe.com/5kQ7sM…`, `bJeeVe…`, `fZu5kE…`) presumably correspond to three pricing tiers. Each is a free-floating URL — if any tier's link is leaked or guessed (the IDs are short Stripe slugs), customers can checkout for the wrong tier. There is no server-side enforcement of tier eligibility.

`portal.mhrmarketing.com` — GoHighLevel exposure

1. **Confirmed GoHighLevel ("HighLevel"/"LeadConnector") whitelabel via `ludicrous.cloud`.** The SaaS-Mode whitelabel pattern means MHR Marketing is the agency tenant; each franchisee gets a sub-account; **MHR retains full admin access to every sub-account.** This is a normal GHL-agency relationship, but it concentrates risk: one MHR admin account compromise = full read/write across all franchisee CRM data, contact lists, marketing automations, and any A2P/SMS credentials.
2. **TinyMCE 4.9.11** (deprecated since 2020) is loaded into the portal page. TinyMCE 4.x has known stored-XSS history and is no longer patched by the upstream. Whether it is exploitable here depends on how GoHighLevel uses it, but loading a known-deprecated rich-text editor in a customer portal is a poor signal about the platform's security hygiene.
3. **The portal is third-party-owned.** Whatever data lives there — leads, contacts, message logs, automation triggers, possibly W-9 / banking / payout info if MHR uses GHL's billing — is in GoHighLevel's databases under MHR's tenancy, governed by GHL's compliance posture, not WaveMAX's. There is no SOC 2 report, no DPA, no breach-notification SLA visible to WaveMAX from this audit.

WordPress hygiene

1. **Admin username `meganbreanne` is publicly disclosed** via `/wp-json/wp/v2/users` and `/?author=1` redirects. This collapses the brute-force search space from "username × password" to just "password."
2. **`/wp-login.php` is publicly reachable** and returns 200 with no rate-limit observable from a single IP. Combined with #1, an attacker can run credential-stuffing or password-spraying against `meganbreanne` directly.
3. **`xmlrpc.php` is reachable** — a classic WordPress amplifier for password-guessing (one HTTP request, hundreds of guesses via `system.multicall`).
4. **No CSP, no `X-Frame-Options`, no `Permissions-Policy`, no `Strict-Transport-Security` on the WP origin** (only Cloudflare's edge policies). The site is clickjackable and embeddable, and any third-party script that gets injected via a plugin XSS runs unconstrained.
5. **No `.well-known/security.txt`** — no published path for security researchers to report findings responsibly.

Risk table (consolidated, severity-ordered)

#	Finding	Severity
1	Intake form submits via **GET** to a public, anonymous Google Apps Script with all data in the URL query string. Logged everywhere; replayable by anyone; no CSRF; no rate limit.	**High**
2	“Payment must be on file” gate is a marketing claim, not a software control. Intake submits without ever touching Stripe.	**High**
3	mhrmarketing.com has no SPF, no DKIM, DMARC `p=none` — Megan’s identity is freely spoofable to franchisees. Given the email of 2026-04-29 trains franchisees to click `mhrmarketing.com` links and enter payment data, this is a phishing setup waiting to happen.	**High**
4	`portal.mhrmarketing.com` is a GoHighLevel whitelabel; MHR holds tenant-wide admin over all franchisee sub-accounts; MHR’s own M365 (GoDaddy-resold, `netorgft2017901`) carries the same legacy-auth weaknesses as WaveMAX. One MHR password phish = total franchisee data exposure.	**High**
5	“Submission successful” modal is hard-coded; `mode:'no-cors'` makes the actual server result invisible. Silent data loss is undetectable.	Medium
6	WordPress admin `meganbreanne` is publicly enumerated via `/wp-json/wp/v2/users` and `/?author=1`. `/wp-login.php` and `/xmlrpc.php` are open.	Medium
7	TinyMCE 4.9.11 (deprecated since 2020, known XSS history) loaded into the portal SPA.	Medium
8	Divi 4.27.6 is current, but Divi has a long advisory history; with no security plugin and no WAF rules visible beyond Cloudflare edge, one stale add-on or a re-introduced Divi vuln takes the whole site.	Medium
9	Photo upload routes to MHR’s externally-owned SharePoint (`netorgft2017901.sharepoint.com`); WaveMAX has no audit trail.	Medium
10	Three free-floating Stripe Payment Links with no tier enforcement; no metadata join between Stripe customer and intake Sheet row. Reconciliation is manual forever.	Medium
11	No CSP / `X-Frame-Options` / `HSTS` / `Permissions-Policy` on the WP origin.	Low
12	Location-slug dropdown ships a `slug-not-found` placeholder that submits as a real value if not changed.	Low
13	No `.well-known/security.txt`; no published responsible-disclosure channel.	Low

What is not broken

- The **payment data flow itself.** Stripe Pricing Table + Payment Links keep card data out of mhrmarketing.com. PCI scope for the merchant is effectively nil. The “card saved but not charged until May 1st” UX is a normal Stripe Setup + scheduled charge.
- **TLS / certificate hygiene.** Both domains have valid Google Trust Services certs.
- **Cloudflare in front of WordPress** is genuinely doing some work — bot challenges on `api.*` and rate-limiting on plugin enumeration kicked in during this audit.

Recommendations to the franchise (re: mhrmarketing.com)

1. **Do not store anything sensitive in the intake form** beyond what's already public marketing copy (location name, hours, public pricing). No SSN, EIN, banking, owner home address, owner DOB, or anything else that could appear on a W-9 should ever go through a Google Apps Script `/exec` endpoint with data in a URL.
2. **Demand from MHR Marketing, in writing:** (a) the Google account that owns the Apps Script and Sheet, and that Sheet's sharing settings; (b) confirmation the GoHighLevel sub-account model has admin access scoped only to MHR staff who need it, with MFA enforced; (c) confirmation that admin access to `mhrmarketing.com` WordPress (`meganbreanne` and any other admins) is MFA-protected and IP-allowlisted; (d) where the photo SharePoint backups live and who has access; (e) MHR's incident-response contact and disclosure timeline commitment.
3. **Require MHR to publish SPF, DKIM, and DMARC on `mhrmarketing.com`** before continuing to use them as a franchise communication channel. Until they do, **any phishing email purporting to be from MeganR@mhrmarketing.com will pass basic checks at receiving providers.**
4. **Replace the intake form** with a real form processor. Options in order of effort: (a) Gravity Forms or Fluent Forms with Stripe integration so the intake is gated by a real payment + webhook (one afternoon of work for a competent WP dev); (b) a Typeform / Tally / HubSpot form with native Stripe integration; (c) an actual Stripe Customer Portal + a webhook that creates the intake record server-side. Any of these gives you (i) a real audit trail, (ii) server-side validation, (iii) a real "paid → intake unlocked" gate, (iv) CSRF protection, (v) verifiable submission receipts.
5. **Spoof-test mhrmarketing.com.** Send a controlled "from MeganR" email from an attacker-perspective server and verify it is rejected by your inbound filters. Right now I expect it to be delivered.
6. **Do not click email links to `mhrmarketing.com` URLs** until SPF/DKIM/DMARC are in place — train franchisees to type the URL directly. This bypasses the spoof vector entirely.

4a. Vibe-Coded Web Presence — Why This Pattern Is Dangerous (Even On WordPress)

> *Vibe coding* is shorthand for someone — often a non-engineer or a junior developer leaning heavily on AI-assisted code generation — assembling a working-looking application by stitching together tutorial snippets, Stack Overflow answers, and LLM output without understanding the full security and operational implications. The mhrmarketing.com intake form is a textbook example. The code runs. It even appears to work. And yet every meaningful security control is either absent, fake, or trivially bypassable.

Why "but it's just WordPress" is not a defense

A common assumption from non-technical site owners is that picking WordPress means they get the platform's security model for free. That isn't how WordPress works. WordPress provides:

- A reasonable core (with timely security releases — assuming you actually apply them).
- A plugin ecosystem with **highly variable** code quality. The average plugin is written by a single developer with no review process; many have had critical CVEs in the last 12 months.
- A theme ecosystem with the same problem (Divi included).

What WordPress does **not** provide:

- Any opinion about how custom JavaScript dropped into a page is structured or what backends it talks to.
- Any review of pasted-in inline scripts before they reach production.
- Any CSP, CSRF, rate-limit, or auth boundary unless you install and configure it yourself.
- Any defense against an admin user uploading code that exfiltrates data to a third-party SaaS.

Once a site owner pastes a 100-line `<script>` block into a Divi page and points it at `script.google.com`, the WordPress security story stops applying. The vulnerable surface is now the developer's homemade pipeline, not WordPress.

The pattern of failure (what to look for in any "we built it ourselves" web work)

These are the warning signs I look for. The mhrmarketing.com intake form hits **every one**:

1. **Single-letter variable names in production JS** (`f`, `U`, `s`, `it`, `nm`, `ok`, `er`). This is what code looks like when it has been minified by hand or ported out of an LLM transcript without review. Real production JS has either readable names or a build pipeline producing source maps. Hand-minified code has no source map, no tests, and no one to fix it when it breaks.
2. **`mode: 'no-cors'` anywhere in form-submit code.** This is a flag that says "I could not solve CORS, so I am hiding the response from myself." It is never the right answer for a production form. It guarantees silent failures.
3. **API endpoints in inline page source.** A real backend has a server-side proxy that hides the third-party endpoint. Pasting `script.google.com/macros/s/...` directly into the HTML means the developer doesn't have a server to put it behind.
4. **Form-submit functions that always show success.** Any code path where the success message fires unconditionally in `.then()` is broken by definition. The whole point of `.then()` vs `.catch()` is to differentiate.

5. **Validation as `alert()`.** Real validation runs both client-side (for UX) and server-side (for trust). A single `alert()` is "I've heard of validation."
6. **Hardcoded "anyone with the link" service URLs** — Google Apps Script `/exec`, Google Forms `/formResponse`, Airtable webhook URLs, Make.com / Zapier hooks, ngrok tunnels in production. These are all fine for prototyping; none of them belong in a customer-facing system without an authenticated proxy in front.
7. **Hand-rolled JS replacing what a $50 plugin solves.** In WordPress specifically, if there is no Gravity Forms / Fluent Forms / Formidable / WPForms in the page source but there is a custom `<script>` doing what those plugins do, someone reinvented the wheel without the spokes (CSRF, captcha, audit log, GDPR consent, server-side validation).
8. **No build process.** No Webpack / Vite / esbuild artifacts, no source maps, no fingerprinted asset filenames. Means the developer doesn't have a development environment separate from production. Means every change is "edit live."
9. **Admin user discoverable via `/wp-json/wp/v2/users` or `/?author=1`.** Means the developer didn't even Google "WordPress harden REST API" once.
10. **No staging environment.** Means there is no place to test changes; every commit is to production.

Why this matters even when nothing has been breached yet

The defects above don't have to be exploited to cause damage. They cause damage by:

- **Failing silently.** Franchisees believe they have submitted intake data; in fact some submissions are being lost to network errors and no one — including MHR — knows which.
- **Eroding trust during incidents.** When the next phishing incident happens, "it came from MHR's portal" is going to be a real question that no one can answer with confidence, because the audit trail is a Google Sheet edited by a script with no logging.
- **Foreclosing future options.** Migrating off the current intake means manually reconciling whatever made it into the Sheet against whatever Stripe says was paid against whatever images made it into MHR's SharePoint. The longer this runs, the harder the cleanup.
- **Setting a precedent.** If this is what got shipped for the *intake* form, the same approach will be applied to every subsequent feature: customer reviews, scheduling, loyalty signups, pickup-and-delivery scheduling, etc. Each one inherits the same defects unless somebody intervenes.

How to ask MHR Marketing to fix it without firing them

You don't have to replace MHR. You do have to set a baseline. A reasonable conversation looks like this:

> "We're glad to have you producing our web presence. Going forward we need any form on our domains to meet a basic bar: a real form processor (Gravity Forms / Fluent Forms / Stripe Customer Portal — pick one and we'll approve it), server-side payment verification before intake unlocks, SPF/DKIM/DMARC published on every domain that sends mail to franchisees, MFA on every admin account on `mhrmarketing.com` and the GoHighLevel agency account, and a written commitment that no franchisee data ever transits a Google Apps Script `/exec` URL. We'll accept a 30-day plan to retrofit the existing intake form to that bar. After that we'd like a quarterly security check-in."

That gives them room to do the right thing without a confrontation, gives WaveMAX a defensible position with franchisees, and creates a paper trail if MHR refuses.

What "good" looks like (a reference baseline for any future MHR-built site)

- Forms processed by a server (a real form plugin, a serverless function, a Stripe webhook), never directly by a `script.google.com` URL.
- Payment-gated workflows enforced at the database level: intake row is created **by a Stripe webhook**, not by a client-side click.
- All admin accounts use phishing-resistant MFA (FIDO2/WebAuthn or Authenticator number-match).
- WP REST `/users` endpoint locked down (the `disable-json-api` / `wp-rest-api-controller` plugins or a `functions.php` filter); `?author=` enumeration redirected; `xmlrpc.php` returned as 403.
- Cloudflare WAF rule blocking `wp-login.php` from anywhere except an admin IP allowlist.
- HSTS, CSP (at least `default-src 'self'; frame-ancestors 'self';`), `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy: strict-origin-when-cross-origin` on every WP origin.
- SPF (`v=spf1 include:<your-mail-provider> -all`), DKIM, DMARC at `p=quarantine` minimum on every sending domain, with rua reports going to a mailbox someone reads.
- A staging environment that mirrors production for changes.
- A short written incident-response plan: who is paged, who notifies WaveMAX, in what timeframe.
- A `/.well-known/security.txt` with a real disclosure email.

None of this is exotic — it is the baseline a competent WordPress agency would consider non-negotiable. Asking for it is reasonable. Accepting less means accepting that the franchise's web presence is one phish away from a bad week.

---

5. Was mhrmarketing.com the credential-harvesting vector for PK?

There is no direct evidence linking the two. But the timeline (phishing inside the tenant within days of franchisees being told to “complete intake + payment + portal login”) is exactly the pattern you’d see if:

- A franchisee user was phished via a fake “MHR portal login” page,
- That user’s M365 password was harvested,
- The credentials worked because either (a) the user reused their wavemaxlaundry.com password on the marketing portal, or (b) the marketing portal was actually authenticating against the M365 tenant (federated single-sign-on).

Investigation pivots:

- Ask MHR Marketing whether `portal.mhrmarketing.com` ever asks for `wavemaxlaundry.com` credentials.
- Check Entra sign-in logs for `pk@wavemaxlaundry.com` for any login originating from a Cloudflare-fronted IP (104.18.x.x / 172.64.x.x range) in the days before the phishing email was sent.
- Cross-reference: did PK ever click an `mhrmarketing.com` link?

---

6. Evidence Captured

DNS / mail authentication

- Authoritative responses from `ns49.domaincontrol.com` for both domains (no MX, no SPF, no DKIM CNAMEs published on either).
- M365 federation realm response for wavemaxlaundry.com showing `NameSpaceType=Federated`, `AuthURL=https://sso.godaddy.com/....`
- DMARC for `wavemaxlaundry.com`: `v=DMARC1; p=quarantine; adkim=r; aspf=r; rua=mailto:dmarc_rua@onsecureserver.net;`.
- DMARC for `mhrmarketing.com`: `v=DMARC1; p=none;`.

Mail headers (phishing email)

- `From: PK <pk@wavemaxlaundry.com>` → `rickh@wavemaxlaundry.com`, 2026-04-29 07:55:47 UTC.
- `X-MS-Exchange-Organization-AuthAs: Internal`, `AuthMechanism: 04`, `AuthSource: PH7PR14MB5641.namprd14.prod.outlook.com`.
- `X-MS-Exchange-CrossTenant-Id: 14e70ab3-eab8-4a93-ab61-b7fec3877261`.
- `Authentication-Results: dkim=none ... dmarc=none action=none`.
- Body is TNEF (`winmail.dat`) with zero-width-char-obfuscated “OneDrive” lure pointing at an `Open` link.

Web stack — mhrmarketing.com

- WordPress 6.9.4 (per `/feed/` `<generator>` tag); Divi 4.27.6 (per `<meta name="generator">`).
- `/readme.html` reachable (HTTP 200).
- `/wp-json/wp/v2/users` discloses admin: id=1, name=`meganbreanne`, slug=`meganbreanne`.
- `/?author=1` 302-redirects to `/author/meganbreanne/`, confirming the slug.
- `/wp-login.php` reachable (HTTP 200, no IP allowlist observed).
- `/xmlrpc.php` reachable.
- Hosting: GoDaddy Managed WordPress (`x-gateway-cache-*`, `x-gateway-request-id` headers), backend IP `160.153.0.197`, fronted by Cloudflare.
- TLS: Google Trust Services, valid 2026-03-12 → 2026-06-10.

Stripe integration (the safe part)

- Pricing table ID: `prctbl_1TJM6nKEqytXvjrU9X3iUyBq`.
- Publishable key: `pk_live_51SzOD8KEqytXvjrUJe5M4sqrFlMvCCExl3AmiFixszYAIpWcmkU0a4R35RdPT3AJxWt19iYuS6GGSUgSCX5sBPG900zqaMziKp` (publishable keys are intended to be public; logging here for inventory).
- Payment Links: `buy.stripe.com/5kQ7sM2zl09FcVi9Nlb3q02`, `buy.stripe.com/bJeeVe7TFcWr6wUgbJb3q01`, `buy.stripe.com/fZu5kEa1Nf4zg7ue3Bb3q03`.
- Pricing tiers visible: $40, $300, $750, $975, $1,050, $1,200, $1,275, $1,500.
- Tier-named “Start with Accelerated” maps to `buy.stripe.com/5kQ7sM2zl09FcVi9Nlb3q02`.

Intake form (`/wavemax-intake/`)

- Apps Script endpoint: `https://script.google.com/macros/s/AKfycbzSEdMVk1UIe3710jIFNBgkBtJw2ZGa6CmDjCIQ2mmGPneKSU50zoR2EVci7fjNHN0/exec`.
- Endpoint deployment: `Anyone, even anonymous` (returns 302 to `script.googleusercontent.com/.../macros/echo` on unauthenticated GET).

- Submit method: `GET` with full payload as `?d=<URL-encoded JSON>`. CORS mode: `no-cors`.
- Field schema (captured from inline JS — full source available in audit working files):
  - `contact`: phone_number, address, city, state, zip, contact_email, google_maps_url
  - `hours`: times, last_wash_time
  - `social`: facebook_url, instagram_url, google_review_url, app_url, pickup_delivery_available, pickup_delivery_price, pickup_delivery_url
  - `pricing`: wdf_price, wdf_minimum, wdf_standard_price, wdf_24_hour_rush_price, google_rating, num_washers, num_dryers, ss_price_min, ss_price_max, ss_dryer_price, ss_dryer_minutes
  - `wdf`: wdf_includes_hangers, wdf_detergent_included, wdf_color_sorting, wdf_same_day_available, promo_bar_text
  - `profile`: owner_name, neighborhood, owner_bio, google_place_id
  - `testimonials`: testimonial_{1,2,3}_{name,text} (rating hardcoded to "5")
  - top-level: wavemax_name, owner_name_top, location_slug, submission_date
- Photo upload destination (per success modal): `https://netorgft2017901.sharepoint.com/sites/ResourceCenter/Photos/Forms/PhotosGrouped.aspx` (MHR's GoDaddy-resold M365 SharePoint).
- Location-slug dropdown values include placeholders `new-location` and `slug-not-found`.

Portal (`portal.mhrmarketing.com` → `whitelabel.ludicrous.cloud`)

- Confirmed **GoHighLevel SaaS-Mode whitelabel**: `class="lead-connector"`, scripts from `static.leadconnectorhq.com`, origin trials for `gohighlevel.com` / `gohighlevel.site`, Pendo analytics tag `2609845c-58c9-4b6e-7126-639c4121d0b7`, Slack app `A04B2S8J7TQ`.
- Loads **TinyMCE 4.9.11** from cdnjs (deprecated 2020; security-EOL).
- TLS SAN: `whitelabel.ludicrous.cloud, *.whitelabel.ludicrous.cloud`.

Phish vector inference

- Both `wavemaxlaundry.com` (WaveMAX franchise) and `netorgft2017901.sharepoint.com` (MHR Marketing) are GoDaddy-resold M365 tenants. Both inherit GoDaddy's federated SSO (`sso.godaddy.com`) identity tier, which historically permits SMTP basic-auth bypass of MFA.
- The phishing email's `AuthMechanism: 04` (SMTP submission) is consistent with that bypass class.

---

7. wavemaxlaundry.com — Public Marketing Site Posture

> **Scope:** the franchise's customer- and prospect-facing marketing site at `https://wavemaxlaundry.com/`. This is a separate concern from the M365 tenant covered in §2/§3 (which deals with mail/identity for `@wavemaxlaundry.com`). The website and the mail tenant share only the domain name; they have different threat models and different fixes. Both are administered indirectly through MHR Marketing's relationship.

Stack identified

- **Hosting:** GoDaddy Managed WordPress (`160.153.0.155`, `x-gateway-cache-*` headers identical to mhrmarketing.com) behind Cloudflare.
- **CMS:** WordPress 6.9.4 (current).
- **Theme:** Divi 5.3.3 (current — *good*; far ahead of mhrmarketing.com's Divi 4.27.6) with a child theme `Divi Child v.5.1.0.1774832092`.
- **Plugin observable in HTML:** `translatepress-multilingual` v3.1.8 (i18n).
- **CRM forms: Zoho CRM Web-to-Contacts** (the "Contact Us" laundry-service form and the "Franchise Inquiry" wizard).
- **Trust badge:** TrustedSite (`cdn.trustedsite.com/js/1.js`) — third-party "secure-site" badge service. Loads a config from `s3-us-west-2.amazonaws.com/mfesecure-public/host/wavemaxlaundry.com/client.json` and a tracker from `cdn.ywxi.net`.
- **Other third-party origins loaded directly into pages:** `ajax.googleapis.com` (jQuery from CDN), `fonts.googleapis.com`, `fonts.gstatic.com`, `crm.zohopublic.com` (Zoho web form analytics servlet), `crm.zoho.com`, `maps.googleapis.com`, `api.zippopotam.us` (zip→city/state lookup), `ipapi.co` (geo-IP), `my.matterport.com` (3D walkthrough iframe), `www.youtube.com` (oEmbed iframe), `www.facebook.com`, `www.instagram.com`, `get.adobe.com`, `img1.wsimg.com` (GoDaddy CDN), and **`jxz.1fc.myftpupload.com`** (a leftover GoDaddy temporary-domain artifact — see §7.2 #6).

7.1 Security headers — none

The site sends **zero security headers** on the home page. Verified directly:

```
HTTP/2 200
server: cloudflare
content-type: text/html; charset=UTF-8
cache-control: public, max-age=2678400
```

```
set-cookie: __cf_bm=...; HttpOnly; Secure
x-gateway-cache-* (GoDaddy)
```

No `Content-Security-Policy`, no `Strict-Transport-Security` (HSTS), no `X-Frame-Options`, no `Referrer-Policy`, no `Permissions-Policy`, no `X-Content-Type-Options`, no `Cross-Origin-*-Policy`. Compare to the affiliate program at `wavemax.promo`, which (per the project handbook) ships strict CSP v2 with nonces and no inline scripts. The franchise marketing site has none of that.

`X-Frame-Options: SAMEORIGIN` *is* present on `/wp-login.php` (set by WordPress core) but not on any other page, so the entire public site is iframe-embeddable on any third-party domain. That enables clickjacking lures against the contact form and the franchise inquiry form.

7.2 Specific implementation problems

1. **WordPress admin user `admin` exists and is publicly enumerated.** `/wp-json/wp/v2/users` returns:

   ```
   id=1, slug=admin,         display="admin",          link=/author/admin/
   id=2, slug=wavemaxadmin,  display="Megan Regalado", link=/author/wavemaxadmin/
   ```

   `?author=1` and `?author=2` 302-redirect to those slugs, confirming both. **`admin` is the default WordPress admin name** and is *the* most password-sprayed username on the internet. Combined with no rate-limiting plugin observed and `/wp-login.php` reachable from anywhere (Cloudflare returns 200, no IP allowlist), this is a brute-force / credential-stuffing target.

2. **No security plugin observable.** Ran a bake-off against the readme paths of Wordfence, Sucuri, AIOS, iThemes Security, Really Simple SSL, Limit Login Attempts, Loginizer, WP-Cerber, NinjaFirewall — none responded. Cloudflare is doing the heavy lifting (it *does* block `/xmlrpc.php` with a 403, which is good), but there is no in-WP defense in depth.

3. **WordPress REST API exposes content endpoints unauthenticated** (`/wp-json/wp/v2/users`, `/posts`, `/pages`, `/comments`). The `users` disclosure is the actively harmful one; the others are typical for a public blog.

4. **Contact form (`#formLaundry`) submits to Zoho CRM Web-to-Contacts** with org-level identifiers embedded in the page source:

   ```html
   <input type="hidden" name="xnQsjsdp" value="4583fa76...20c295eac762...">
   <input type="hidden" name="xmIwtLD"  value="9d1e13b7...93d83424...">
   <input type="hidden" name="actionType" value="Q29udGFjdHM=">  <!-- "Contacts" -->
   <input type="hidden" name="aG9uZXlwb3Q" value="">              <!-- "honeypot" -->
   ```

   These hidden tokens are *Zoho's documented public web-form identifiers*, not secrets — but they identify your Zoho org and your "Contacts" target module. **Anyone can copy the form to their own server and inject arbitrary records into your Zoho Contacts module.** Mitigations Zoho expects you to enable: (a) a Zoho-managed reCAPTCHA on the web form (not present here), (b) the `aG9uZXlwb3Q` honeypot (present, empty — works against the laziest bots only), (c) Zoho-side spam rules. The form has **no visible CAPTCHA** at all, which means the only barrier between an attacker and your CRM is one base64-named honeypot field. Expect this to be filled with junk Contacts on a regular basis.

5. **Franchise inquiry form (`#formFranchise`) collects** financial qualification data** (cash available up to "$1M+", net worth up to "$2M+") and submits the same way to Zoho.** Same anti-abuse posture — no CAPTCHA, just the honeypot. Worse: an attacker who scrapes inbound submissions (via, e.g., email side-channel from a compromised Megan account, see §1) gets **a list of qualified high-net-worth franchise prospects** with phone, email, and stated liquid capital. This is high-value reconnaissance data for follow-on social engineering.

6. **A leftover GoDaddy temporary-hosting domain is referenced in the page source: `https://jxz.1fc.myftpupload.com`.** GoDaddy issues `*.myftpupload.com` URLs as temporary placeholder domains during site setup. Finding one in production HTML means either (a) the WordPress database still contains hardcoded references to the old staging URL (a botched migration), or (b) some asset is still being served from the staging copy. Either way it's a leak: the staging URL likely runs the same site without the same Cloudflare protection, so an attacker can fetch the same content directly from GoDaddy's edge and bypass any Cloudflare WAF rules you eventually add. Search the WordPress DB and `wp-content/uploads/` for `myftpupload.com` and rewrite to the canonical URL with WP-CLI's `search-replace`.

7. **TrustedSite badge loads third-party JS that runs on every page.** `cdn.trustedsite.com/js/1.js` then loads `client.json` from a public S3 bucket and trackers from `cdn.ywxi.net`. The script is benign as long as TrustedSite is uncompromised, but it does have full DOM access — it can read the contact-form fields before submit if it ever turned malicious or got hijacked at the CDN. With no CSP, there's no defense-in-depth here.

8. **jQuery is loaded from `ajax.googleapis.com` instead of the WordPress origin.** Mixing CDN-hosted jQuery with theme-bundled jQuery 3.7.1 from `wp-includes/js/jquery/` is a code-smell suggesting two different developers added scripts independently. With no CSP, any CDN compromise (or DNS hijack) executes in your origin's context and can read both forms.

9. **No `.well-known/security.txt`** — no published path for security researchers to report findings responsibly.

10. **The site is iframe-embeddable** (no `X-Frame-Options`, no `frame-ancestors` CSP). A clickjacking lure that overlays an invisible WaveMAX franchise-inquiry form on an attacker-controlled page can capture qualified prospects without their knowledge.

11. **Cookies are minimal but Cloudflare's `__cf_bm` is set on every request** with `HttpOnly; Secure`. No PII in cookies observed. WordPress login cookies will appear when authenticated — those should also be `Secure` and have `SameSite=Strict` set; not testable without credentials.

12. **No HSTS preload.** The domain is not on the HSTS preload list. Combined with the absent `Strict-Transport-Security` header, a downgrade-to-HTTP attack on a coffee-shop / hotel wifi is possible against a first-time visitor. This particularly matters for users entering financial-qualification data into the franchise form.

7.3 Hardening plan (priority order)

Within 1 day (free, low risk)

1. **Lock down the WP REST `users` endpoint.** Either disable the `wp/v2/users` route for unauthenticated requests via a `functions.php` filter on `rest_endpoints`, or install a hardening plugin (Wordfence / Solid Security / Disable WP REST API) and enable the "users endpoint" rule. **Specifically: rename the `admin` user (id=1) to something non-default, then disable the endpoint.** Renaming requires a DB-level change (`wp user update 1 --user_login=...` via WP-CLI) since WP doesn't allow it through the UI.

2. **Block `?author=N` enumeration** via a 301 to `/` for anonymous users, or via the same security plugin.

3. **Add a Cloudflare WAF rule allowing `/wp-login.php` only from an MHR Marketing admin IP allowlist.** This single rule eliminates 99% of credential-spray traffic.

4. **Add Cloudflare Page Rules / Transform Rules to set security headers at the edge** — no WordPress changes needed:

   ```
   Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
   X-Frame-Options: SAMEORIGIN
   X-Content-Type-Options: nosniff
   Referrer-Policy: strict-origin-when-cross-origin
   Permissions-Policy: geolocation=(), camera=(), microphone=(), payment=()
   ```

5. **Search-replace the `jxz.1fc.myftpupload.com` references out of the WordPress database.** WP-CLI: `wp search-replace 'jxz.1fc.myftpupload.com' 'wavemaxlaundry.com' --all-tables --dry-run` first, then drop `--dry-run`.

Within 1 week (small effort, real lift)

6. **Enable Zoho's reCAPTCHA on both web forms.** This is a config-only change in Zoho's CRM Setup → Automation → Web Forms; the form re-publishes with a CAPTCHA token requirement and the existing markup updates. Without it, both forms are spam magnets and the franchise-qualification form is a reconnaissance gift.

7. **Replace the `admin`/id=1 user.** Create a new admin with a non-obvious username, transfer ownership of all id=1 posts to the new user, change id=1's role to *Subscriber* (not delete — preserves authorship integrity for old content), and revoke its login by setting an unguessable password it doesn't know.

8. **MFA on every WordPress admin account** — both `admin` (after renaming) and `wavemaxadmin`. Either Wordfence Login Security (free plugin, TOTP) or a Cloudflare Zero Trust enforcement layer in front of `/wp-login.php` and `/wp-admin/`. Verify Megan's account specifically — she is identifiable by name in the user enumeration, so spear-phishing is trivial and the MHR-side ATO already happened on `mhrmarketing.com` (see §1, §4). If she reuses a password, both sites fall together.

9. **Add a content security policy.** Start with a *report-only* policy that allows current third parties, then tighten:

   ```
   Content-Security-Policy-Report-Only: default-src 'self'; script-src 'self' 'unsafe-inline'
   https://ajax.googleapis.com https://cdn.trustedsite.com https://cdn.ywxi.net https://crm.zohopublic.com
   https://crm.zoho.com https://maps.googleapis.com https://www.google.com https://www.youtube.com; style-src
   'self' 'unsafe-inline' https://fonts.googleapis.com; img-src 'self' data: https:; font-src 'self'
   https://fonts.gstatic.com; connect-src 'self' https://api.zippopotam.us https://ipapi.co https://crm.zoho.com
   https://maps.googleapis.com; frame-src https://my.matterport.com https://www.youtube.com
   https://www.facebook.com; frame-ancestors 'self'; report-uri /csp-report;
   ```

   Read the report endpoint for a week, then move to enforcing mode and progressively drop `'unsafe-inline'` (which will require fixing inline `<style>` blocks in Divi and inline event handlers like `onclick="franchiseNext(0)"` in the franchise form — Divi has a CSP option in its theme settings).

10. **Disable XML-RPC at the WordPress level** (not just at Cloudflare). A `functions.php` filter: `add_filter('xmlrpc_enabled', '__return_false');`. Belt-and-suspenders against the day someone bypasses Cloudflare.

Within 1 month (structural)

11. **Move WordPress admin behind Cloudflare Zero Trust** (formerly Cloudflare Access). Free for up to 50 users. Removes `/wp-login.php` and `/wp-admin/` from the public internet entirely; admins authenticate to Cloudflare first (with MFA), then reach

WordPress.

12. **Stand up a staging environment** under a private subdomain (`staging-wavemaxlaundry.example`, IP-restricted via Cloudflare) so any future MHR work — or your own work — has somewhere to test before hitting production.
13. **Quarterly WP/plugin/theme update cadence** with a documented owner. WordPress core, Divi, Divi Child, TranslatePress, and any new plugin must all be inside their support window. The Divi version on this site (5.3.3) is current today; that's a decision to maintain, not an achievement.
14. **Publish a `/.well-known/security.txt`** with a real disclosure address. Researchers will report findings to you instead of the bug-bounty grey market or, worse, dropping them publicly.
15. **Commission an independent WordPress audit** when MHR delivers their next major change. If MHR is going to be the long-term web vendor (per §4 framing), you need someone outside that relationship to spot-check the work — at minimum once per major release.

7.4 Risk table (consolidated)

#	Finding	Severity
1	WP user `admin` (id=1) exists and is publicly enumerated; `/wp-login.php` is open with no rate-limit plugin and no IP allowlist. Password-spray ready.	**High**
2	No CSP, no HSTS, no Permissions-Policy, no X-Frame-Options on the public site. Site is clickjackable; first-visit downgrade attack possible.	**High**
3	Franchise inquiry form collects high-value financial-qualification data (cash up to $1M+, net worth up to $2M+) and submits to Zoho with no CAPTCHA — both spam target and reconnaissance asset.	**High**
4	Both Zoho forms have no CAPTCHA — Contacts module is spammable into uselessness.	Medium
5	Leftover `jxz.1fc.myftpupload.com` GoDaddy staging domain referenced in production HTML — possible Cloudflare bypass to origin.	Medium
6	Megan Regalado (`wavemaxadmin`) is identified in user enumeration and is also a target on `mhrmarketing.com`; password reuse closes both.	Medium
7	TrustedSite + Google CDN jQuery + Zoho analytics all execute with full DOM access; with no CSP, any one CDN compromise is total.	Medium
8	Site is iframe-embeddable; clickjacking-overlay attack possible against the franchise form.	Medium
9	No `.well-known/security.txt`.	Low
10	No staging environment observable; "edit live" is the implied workflow.	Low

7.5 What is in good shape

- **WordPress core, Divi, jQuery are all current** — a positive signal about whoever's keeping the site updated.
- **Cloudflare is doing real work** — `/xmlrpc.php` is blocked at the edge, `/.env` and `/wp-config.php.bak` return 403, `__cf_bm` is set with `HttpOnly; Secure`.
- **Zoho web forms are the right architectural choice** for marketing leads — far better than the Apps Script anti-pattern at mhrmarketing.com. The implementation just needs CAPTCHA enabled and the public form-token model accepted as "abuse-prone by design."
- **The TrustedSite badge has been there long enough to be configured** (the script fetches an org-specific config), suggesting this site has had at least *some* security ownership, even if it's not deep.

The contrast with `mhrmarketing.com` is sharp: `wavemaxlaundry.com` shows signs of a competent if minimal WordPress administrator (current versions, Cloudflare WAF tuned, real CRM integration), while `mhrmarketing.com` shows signs of someone iterating live on a

production page with hand-rolled JS. Holding MHR's future work for `wavemaxlaundry.com` to the bar set by `wavemaxlaundry.com`'s own current state — not by `mhrmarketing.com`'s — is reasonable.

Generated 2026-04-30. Update this file as remediation progresses.